

第3章 数组和字符串

学习目标

- ☑ 掌握一维数组的声明和使用方法
- ☑ 掌握二维数组的声明和使用方法
- ☑ 掌握字符串的声明、赋值、比较和连接方法
- ☑ 熟悉字符的 ASCII 码和 `ctype.h` 中的字符函数
- ☑ 正确认识“++”、“+=”等能修改变量的运算符
- ☑ 掌握 `fgetc` 和 `getchar` 的使用方法
- ☑ 了解不同操作系统中换行符的表示方法
- ☑ 掌握 `fgets` 的使用方法并了解 `gets` 的“缓冲区溢出”漏洞
- ☑ 学会用常量表简化代码

第2章的程序很实用，也发挥出了计算机的计算优势，但没有发挥出计算机的存储优势——我们只用了屈指可数的变量。尽管有的程序也处理了大量的数据，但这些数据都只是“过客”，只参与了计算，并没有被保存下来。

本章介绍数组和字符串，二者都能保存大量的数据。字符串是一种数组（字符数组），但由于其应用的特殊性，适用一些特别的处理方式。

3.1 数 组

考虑这样一个问题：读入一些整数，逆序输出到一行中。已知整数不超过 100 个。如何编写这个程序呢？首先是循环读取输入。读入每个整数以后，应该做些什么呢？思来想去，在所有整数全部读完之前，似乎没有其他事可做。换句话说，只能把每个数都存下来。存放在哪里呢？答案是：数组。

程序 3-1 逆序输出

```
#include <stdio.h>
#define maxn 105
int a[maxn];
int main()
{
    int x, n = 0;
    while (scanf("%d", &x) == 1)
        a[n++] = x;
    for (int i = n - 1; i >= 0; i--)
```

```

    printf("%d ", a[i]);
    printf("%d\n", a[0]);
    return 0;
}

```

语句“`int a[maxn]`”声明了一个包含 `maxn` 个整型变量的数组，它们是：`a[0]`, `a[1]`, `a[2]`, ..., `a[maxn-1]`。注意，没有 `a[maxn]`。

提示 3-1：语句“`int a[maxn]`”声明了一个包含 `maxn` 个整型变量的数组，即 `a[0]`, `a[1]`, ..., `a[maxn-1]`，但不包含 `a[maxn]`。`maxn` 必须是常数，不能是变量。

为什么这里声明 `maxn` 为 105 而不是 100 呢？因为这样更保险。

提示 3-2：在算法竞赛中，常常难以精确计算出需要的数组大小，数组一般会声明得稍大一些。在空间够用的前提下，浪费一点不会有太大影响。

接下来是语句“`a[n++] = x`”，它做了两件事：首先赋值 `a[n]=x`，然后执行 `n=n+1`。如果觉得难以理解，可以将其改写成“`{ a[n] = x; n=n+1; }`”。注意这里的花括号是不能省略的，因为在默认情况下，`for` 语句的循环体只有一条语句。只有使用花括号时，花括号里的语句才会整体作为循环体。一般地，当表达式里出现 `n++` 时，表达式会使用加 1 前的 `n` 计算表达式，当表达式计算完毕之后再给 `n` 加 1。

和 `n++` 相对应的，还有一个 `++n`，表示先给 `n` 增加 1，然后使用新的 `n`。前缀和后缀“++”运算符是 C 语言的特色之一。事实上，后面将要介绍的 C++ 语言名字里的“++”就是该运算符。

提示 3-3：对于变量 `n`，`n++` 和 `++n` 都会给 `n` 加 1，但当它们用在一个表达式中时，行为有所差别：`n++` 会使用加 1 前的值计算表达式，而 `++n` 会使用加 1 后的值计算表达式。“++”运算符是 C 语言的特色之一。

循环结束后，数据被存储在 `a[0]`, `a[1]`, ..., `a[n-1]` 中，其中变量 `n` 是整数的个数（想一想，为什么）。

存好以后就可以输出了：依次输出 `a[n-1]`, `a[n-2]`, ..., `a[1]` 和 `a[0]`。这里有一个小问题：一般要求输出的行首行尾均无空格，相邻两个数据间用单个空格隔开。这样，一共要输出 `n` 个整数，但只有 `n-1` 个空格，所以只好分两条语句输出。

在上述程序中，数组 `a` 被声明在 `main` 函数的外面。请试着把 `maxn` 定义中的 100 改成 1000000，比较一下把数组 `a` 放在 `main` 函数内外的运行结果是否相同。如果相同，试着把 1000000 改得再大一些。当实验完成之后，读者应该就能明白为什么要把 `a` 的定义放在 `main` 函数的外面了。简单地说，只有在放外面时，数组 `a` 才可以开得很大；放在 `main` 函数内时，数组稍大就会异常退出。其道理将在后面讨论，现在只需要记住规则即可。

提示 3-4：比较大的数组应尽量声明在 `main` 函数外，否则程序可能无法运行。

C 语言的数组并不是“一等公民”，而是“受歧视”的。例如，数组不能够进行赋值操作：在程序 3-1 中，如果声明的是“`int a[maxn], b[maxn]`”，是不能赋值 `b = a` 的。如果要从

数组 a 复制 k 个元素到数组 b ，可以这样做：`memcpy(b, a, sizeof(int) * k)`。当然，如果数组 a 和 b 都是浮点型的，复制时要写成“`memcpy(b, a, sizeof(double) * k)`”。另外需要注意的是，使用 `memcpy` 函数要包含头文件 `string.h`。如果需要把数组 a 全部复制到数组 b 中，可以写得简单一些：`memcpy(b, a, sizeof(a))`。

开灯问题。有 n 盏灯，编号为 $1 \sim n$ 。第 1 个人把所有灯打开，第 2 个人按下所有编号为 2 的倍数的开关（这些灯将被关掉），第 3 个人按下所有编号为 3 的倍数的开关（其中关掉的灯将被打开，开着的灯将被关闭），依此类推。一共有 k 个人，问最后有哪些灯开着？输入 n 和 k ，输出开着的灯的编号。 $k \leq n \leq 1000$ 。

样例输入：

7 3

样例输出：

1 5 6 7

【分析】

用 $a[1], a[2], \dots, a[n]$ 表示编号为 $1, 2, 3, \dots, n$ 的灯是否开着。模拟这些操作即可。

程序 3-2 开灯问题

```
#include<stdio.h>
#include<string.h>
#define maxn 1010
int a[maxn];
int main()
{
    int n, k, first = 1;
    memset(a, 0, sizeof(a));
    scanf("%d%d", &n, &k);
    for(int i = 1; i <= k; i++)
        for(int j = 1; j <= n; j++)
            if(j % i == 0) a[j] = !a[j];
    for(int i = 1; i <= n; i++)
        if(a[i]) { if(first) first = 0; else printf(" "); printf("%d", i); }
    printf("\n");
    return 0;
}
```

“`memset(a, 0, sizeof(a))`”的作用是把数组 a 清零，它也在 `string.h` 中定义。虽然也能用 `for` 循环完成相同的任务，但是用 `memset` 又方便又快捷。另一个技巧在输出：为了避免输出多余空格，设置了一个标志变量 `first`，可以表示当前要输出的变量是否为第一个。第一个变量前不应有空格，但其他变量都有。

蛇形填数。在 $n \times n$ 方阵里填入 $1, 2, \dots, n \times n$ ，要求填成蛇形。例如， $n=4$ 时方阵为：

```

10 11 12 1
9 16 13 2
8 15 14 3
7 6 5 4

```

上面的方阵中, 多余的空格只是为了便于观察规律, 不必严格输出。 $n \leq 8$ 。

【分析】

类比数学中的矩阵, 可以用一个二维数组来储存题目中的方阵。只需声明一个“`int a[maxn][maxn]`”, 就可以获得一个大小为 $\text{maxn} \times \text{maxn}$ 的方阵。在声明时, 二维的大小不必相同, 因此也可以声明 `int a[30][50]` 这样的数组, 第一维下标范围是 $0, 1, 2, \dots, 29$, 第二维下标范围是 $0, 1, 2, \dots, 49$ 。

提示 3-5: 可以用“`int a[maxn][maxm]`”生成一个整型的二维数组, 其中 maxn 和 maxm 不必相等。这个数组共有 $\text{maxn} \times \text{maxm}$ 个元素, 分别为 `a[0][0]`, `a[0][1]`, $\dots$, `a[0][maxm-1]`, `a[1][0]`, `a[1][1]`, $\dots$, `a[1][maxm-1]`, $\dots$, `a[maxn-1][0]`, `a[maxn-1][1]`, $\dots$, `a[maxn-1][maxm-1]`。

从 1 开始依次填写。设“笔”的坐标为 (x, y) , 则一开始 $x=0$, $y=n-1$, 即第 0 行, 第 $n-1$ 列 (行列的范围是 $0 \sim n-1$, 没有第 n 列)。“笔”的移动轨迹是: 下, 下, 下, 左, 左, 左, 上, 上, 上, 右, 右, 下, 下, 左, 上。总之, 先是下, 到不能填为止, 然后是左, 接着是上, 最后是右。“不能填”是指再走就出界 (例如 $4 \rightarrow 5$), 或者再走就要走到以前填过的格子 (例如 $12 \rightarrow 13$)。如果把所有格子初始化为 0, 就能很方便地加以判断。

程序 3-3 蛇形填数

```

#include<stdio.h>
#include<string.h>
#define maxn 20
int a[maxn][maxn];
int main()
{
    int n, x, y, tot = 0;
    scanf("%d", &n);
    memset(a, 0, sizeof(a));
    tot = a[x=0][y=n-1] = 1;
    while(tot < n*n)
    {
        while(x+1<n && !a[x+1][y]) a[++x][y] = ++tot;
        while(y-1>=0 && !a[x][y-1]) a[x][--y] = ++tot;
        while(x-1>=0 && !a[x-1][y]) a[--x][y] = ++tot;
        while(y+1<n && !a[x][y+1]) a[x][++y] = ++tot;
    }
    for(x = 0; x < n; x++)
    {

```